

# lab05

## Parte 1

```
library(readr)
```

Warning: pacote 'readr' foi compilado no R versão 4.5.3

```
library(dplyr)
```

Warning: pacote 'dplyr' foi compilado no R versão 4.5.3

Anexando pacote: 'dplyr'

Os seguintes objetos são mascarados por 'package:stats':

filter, lag

Os seguintes objetos são mascarados por 'package:base':

intersect, setdiff, setequal, union

```
library(ggplot2)
```

Warning: pacote 'ggplot2' foi compilado no R versão 4.5.3

```
library(maps)
```

Warning: pacote 'maps' foi compilado no R versão 4.5.3

```
library(stringr)
```

Warning: pacote 'stringr' foi compilado no R versão 4.5.3

**a)**

```
analisa_aeronave <- function(tail_number, arquivo) {  
  # O parâmetro 'tail_number' recebe a identificação da aeronave (ex: "N431WN")  
  # O parâmetro 'arquivo' recebe o caminho para o arquivo zipado ou CSV  
  message("Iniciando análise para a aeronave: ", tail_number)  
}
```

**b)**

```
carregar_dados_aeronave <- function(tail_number, arquivo) {  
  # Função interna de filtro por bloco  
  filtrar_aeronave <- function(chunk, pos) {  
    chunk %>% filter(TAIL_NUMBER == !!tail_number)  
  }  
  # Leitura por partes de 1 milhão de linhas  
  tabela_bruta <- read_csv_chunked(  
    file = arquivo,  
    callback = DataFrameCallback$new(filtrar_aeronave),  
    chunk_size = 1000000,  
    show_col_types = FALSE  
  )  
  # Ordenação cronológica mantendo todas as colunas  
  tabela_tidy <- tabela_bruta %>%  
    arrange(YEAR, MONTH, DAY, DEPARTURE_TIME)  
  
  return(tabela_tidy)  
}
```

c)

```
gerar_mapa_trajetoria <- function(tabela_tidy, tail_number, arquivo_airports = "airports.csv") {
  # Carregar aeroportos com as colunas necessárias
  airports <- read_csv(
    arquivo_airports,
    col_select = c(IATA_CODE, LATITUDE, LONGITUDE),
    show_col_types = FALSE
  )

  # Criar uma coluna indicando a ordem temporal do voo (1, 2, 3...) e juntar (join) a latitud
  dados_geograficos <- tabela_tidy %>%
    mutate(ordem_voo = row_number()) %>%
    # Adiciona Latitude/Longitude da ORIGEM
    left_join(airports, by = c("ORIGIN_AIRPORT" = "IATA_CODE")) %>%
    rename(lat_orig = LATITUDE, lon_orig = LONGITUDE) %>%
    # Adiciona Latitude/Longitude do DESTINO
    left_join(airports, by = c("DESTINATION_AIRPORT" = "IATA_CODE")) %>%
    rename(lat_dest = LATITUDE, lon_dest = LONGITUDE) %>%
    # Remove voos sem coordenadas mapeadas
    filter(!is.na(lat_orig), !is.na(lat_dest))

  # Carregar o contorno dos estados dos EUA para o fundo
  mapa_eua <- map_data("state")

  # Construção do gráfico com ggplot2
  mapa_grafico <- ggplot() +
    # Desenha o mapa dos EUA ao fundo
    geom_polygon(
      data = mapa_eua,
      aes(x = long, y = lat, group = group),
      fill = "#f2f2f2",
      color = "#ffffff"
    ) +
    # Segmentos/Linhas que ligam a origem ao destino do voo
    geom_segment(
      data = dados_geograficos,
      aes(
        x = lon_orig,
        y = lat_orig,
        xend = lon_dest,
        yend = lat_dest,
        color = ordem_voo # Cor indicando a ordem temporal dos voos
      )
    )
}
```

```

    ),
    linewidth = 0.7,
    alpha = 0.7
  ) +
  # Pontos representando os aeroportos
  geom_point(
    data = dados_geograficos,
    aes(x = lon_orig, y = lat_orig),
    color = "black",
    size = 1.2
  ) +
  # Paleta de cor contínua para mostrar o avanço do tempo
  scale_color_viridis_c(option = "magma", name = "Ordem dos Voos") +
  # Projeção espacial limitada aos EUA continentais
  coord_quickmap(xlim = c(-125, -66), ylim = c(24, 50)) +
  labs(
    title = paste("Trajetória Anual do Avião:", tail_number),
    x = "Longitude",
    y = "Latitude"
  ) +
  theme_minimal()

return(mapa_grafico)
}

```

d)

```

# Adiciona a estatística descritiva (velocidade) e refina o mapa
gerar_mapa_com_estatistica <- function(tabela_tidy, tail_number, arquivo_airports = "airports") {

  # Carrega as coordenadas dos aeroportos
  airports <- read_csv(
    arquivo_airports,
    col_select = c(IATA_CODE, LATITUDE, LONGITUDE),
    show_col_types = FALSE
  )

  # Prepara os dados geográficos e calcula a estatística descritiva
  # Estatística escolhida: Velocidade Média do voo em mph (Distância / Tempo de voo * 60)
  dados_geograficos <- tabela_tidy %>%

```

```

mutate(ordem_voo = row_number()) %>%
left_join(airports, by = c("ORIGIN_AIRPORT" = "IATA_CODE")) %>%
rename(lat_orig = LATITUDE, lon_orig = LONGITUDE) %>%
left_join(airports, by = c("DESTINATION_AIRPORT" = "IATA_CODE")) %>%
rename(lat_dest = LATITUDE, lon_dest = LONGITUDE) %>%
filter(!is.na(lat_orig), !is.na(lat_dest)) %>%
# Cálculo da estatística descritiva de interesse
mutate(velocidade_media = if_else(!is.na(AIR_TIME) & AIR_TIME > 0, (DISTANCE / AIR_TIME)

# Base do mapa dos EUA
mapa_eua <- map_data("state")

# Construção do gráfico mapeando a estatística no elemento estético (linewidth)
mapa_grafico <- ggplot() +
  geom_polygon(
    data = mapa_eua,
    aes(x = long, y = lat, group = group),
    fill = "#f8f9fa",
    color = "#d1d5db"
  ) +
  # Mapeia 'velocidade_media' na espessura da linha (linewidth)
  geom_segment(
    data = dados_geograficos,
    aes(
      x = lon_orig, y = lat_orig,
      xend = lon_dest, yend = lat_dest,
      color = ordem_voo,
      # Mantém a sequência temporal
      linewidth = velocidade_media
      # Estética associada à estatística descritiva
    ),
    alpha = 0.65
  ) +
  geom_point(
    data = dados_geograficos,
    aes(x = lon_orig, y = lat_orig),
    color = "#1f2937",
    size = 1.1
  ) +
  scale_color_viridis_c(option = "turbo", name = "Ordem dos Voos") +
  scale_linewidth_continuous(range = c(0.3, 1.8), name = "Velocidade (mph)") +
  coord_quickmap(xlim = c(-125, -66), ylim = c(24, 50)) +

```

```

    labs(
      title = paste("Trajetória e Velocidade Média - Aeronave:", tail_number),
      x = "Longitude",
      y = "Latitude"
    ) +
    theme_minimal()

  return(mapa_grafico)
}

```

## Final

```

analisa_aeronave <- function(tail_number, arquivo_flights = "flights.csv", arquivo_airports = "airports.csv") {
  # ITEM A e B: Leitura em blocos de 1 milhão, filtragem por aeronave e ordenação cronológica
  # Função callback para filtrar apenas a aeronave desejada durante a leitura
  filtrar_aeronave <- function(chunk, pos) {
    chunk %>% filter(TAIL_NUMBER == !!tail_number)
  }

  # Leitura eficiente em blocos de 1 milhão de linhas
  tabela_bruta <- read_csv_chunked(
    file = arquivo_flights,
    callback = DataFrameCallback$new(filtrar_aeronave),
    chunk_size = 1000000,
    show_col_types = FALSE
  )

  # Ordenação temporal cronológica preservando todas as colunas
  tabela_tidy <- tabela_bruta %>%
    arrange(YEAR, MONTH, DAY, DEPARTURE_TIME)

  # ITEM C e D: Cruzamento de coordenadas geográficas, cálculo da estatística descritiva e geográfica
  # Leitura do arquivo de aeroportos
  airports <- read_csv(
    arquivo_airports,
    col_select = c(IATA_CODE, LATITUDE, LONGITUDE),
    show_col_types = FALSE
  )

  # Preparação dos dados espaciais e cálculo da estatística (Item D: Velocidade Média em mph)
}

```

```

tabela_mapa <- tabela_tidy %>%
  mutate(ordem_voo = row_number()) %>%
  left_join(airports, by = c("ORIGIN_AIRPORT" = "IATA_CODE")) %>%
  rename(lat_orig = LATITUDE, lon_orig = LONGITUDE) %>%
  left_join(airports, by = c("DESTINATION_AIRPORT" = "IATA_CODE")) %>%
  rename(lat_dest = LATITUDE, lon_dest = LONGITUDE) %>%
  filter(!is.na(lat_orig), !is.na(lat_dest)) %>%
  # ITEM D: Estatística de interesse -> Velocidade média (Distância / Tempo * 60)
  mutate(velocidade_media = if_else(!is.na(AIR_TIME) & AIR_TIME > 0, (DISTANCE / AIR_TIME)

# Contorno geográfico dos EUA
mapa_eua <- map_data("state")

# Construção do gráfico de trajetória com ggplot2
mapa_grafico <- ggplot() +
  geom_polygon(
    data = mapa_eua,
    aes(x = long, y = lat, group = group),
    fill = "#f8f9fa",
    color = "#d1d5db"
  ) +
  # ITEM C (color = ordem_voo) e ITEM D (linewidth = velocidade_media)
  geom_segment(
    data = tabela_mapa,
    aes(
      x = lon_orig, y = lat_orig,
      xend = lon_dest, yend = lat_dest,
      color = ordem_voo,
      # ITEM C: Sequência temporal
      linewidth = velocidade_media
      # ITEM D: Estética mapeada à estatística descritiva
    ),
    alpha = 0.65
  ) +
  geom_point(
    data = tabela_mapa,
    aes(x = lon_orig, y = lat_orig),
    color = "#1f2937",
    size = 1.1
  ) +
  scale_color_viridis_c(option = "turbo", name = "Ordem dos Voos") +
  scale_linewidth_continuous(range = c(0.3, 1.8), name = "Velocidade Média (mph)") +

```

```

coord_quickmap(xlim = c(-125, -66), ylim = c(24, 50)) +
labs(
  title = paste("Trajetória e Velocidade da Aeronave:", tail_number),
  x = "Longitude",
  y = "Latitude"
) +
theme_minimal()

# Retorno dos dois elementos solicitados (Tabela Tidy + Gráfico)
return(list(
  tabela = tabela_tidy,
  mapa = mapa_grafico
))
}

```

## Teste

```

teste <- analisa_aeronave(
  tail_number = "N431WN",
  arquivo_flights = "flights.csv",
  arquivo_airports = "airports.csv"
)
head(teste$tabela)

```

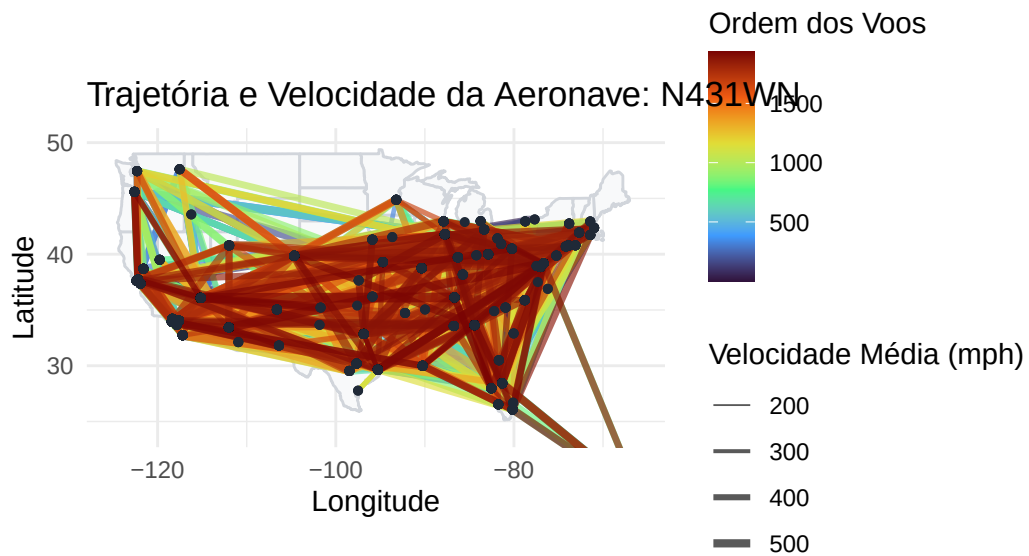
```

# A tibble: 6 x 31
  YEAR MONTH DAY DAY_OF_WEEK AIRLINE FLIGHT_NUMBER TAIL_NUMBER ORIGIN_AIRPORT
  <dbl> <dbl> <dbl>         <dbl> <chr>          <dbl> <chr>          <chr>
1  2015     1     1             4 WN             1762 N431WN        DCA
2  2015     1     1             4 WN             3564 N431WN        HOU
3  2015     1     1             4 WN             3564 N431WN        BOS
4  2015     1     1             4 WN             3564 N431WN        BWI
5  2015     1     2             5 WN             1793 N431WN        MCI
6  2015     1     2             5 WN             1793 N431WN        LAX
# i 23 more variables: DESTINATION_AIRPORT <chr>, SCHEDULED_DEPARTURE <chr>,
# DEPARTURE_TIME <chr>, DEPARTURE_DELAY <dbl>, TAXI_OUT <dbl>,
# WHEELS_OFF <chr>, SCHEDULED_TIME <dbl>, ELAPSED_TIME <dbl>, AIR_TIME <dbl>,
# DISTANCE <dbl>, WHEELS_ON <chr>, TAXI_IN <dbl>, SCHEDULED_ARRIVAL <chr>,
# ARRIVAL_TIME <chr>, ARRIVAL_DELAY <dbl>, DIVERTED <dbl>, CANCELLED <dbl>,
# CANCELLATION_REASON <chr>, AIR_SYSTEM_DELAY <dbl>, SECURITY_DELAY <dbl>,
# AIRLINE_DELAY <dbl>, LATE_AIRCRAFT_DELAY <dbl>, WEATHER_DELAY <dbl>

```

```
print(teste$mapa)
```

Warning: Removed 16 rows containing missing values or values outside the scale range (``geom_segment()``).



```
Sys.setenv(TZ = "America/Sao_Paulo")  
Sys.time()
```

```
[1] "2026-09-16 17:22:48 -03"
```